

DATA VISUALIZATION

installing matplotlib , seaborn , pandas, numpy,

```
!pip install matplotlib seaborn pandas numpy
```

[Show hidden output](#)

Implementing matplotlib

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

matplotlib → complete visualization library

pyplot → module used for plotting graphs

plt → shortcut name (alias)

```
# Every chart follows this pattern:
# 1. Create figure & axes
# 2. Plot data

# 3. Add labels 4. Show/Save
```

Line Chart — Trends over time

```
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
sales = [1200, 1500, 1100, 1800, 2100, 1950]
costs = [800, 900, 850, 1000, 1100, 1050]
```

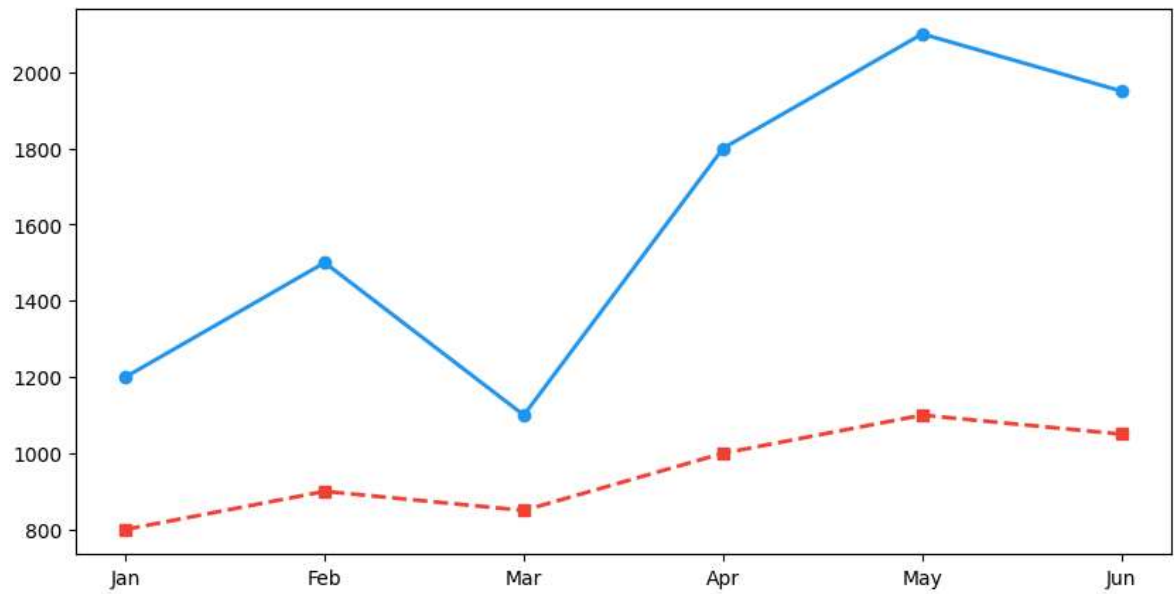
```
plt.figure(figsize=(10, 5))

# width, height in inches
plt.plot(months, sales, marker='o', label='Sales', color='#2196F3', linewidth=2)

# plt.plot is used to draw line graphs
# | Marker | Shape |
# | ----- | ----- |
# | 'o' | circle |
# | 's' | square |
# | '^' | triangle |
# | '*' | star |
# | '+' | plus |
# | 'x' | x shape |
```

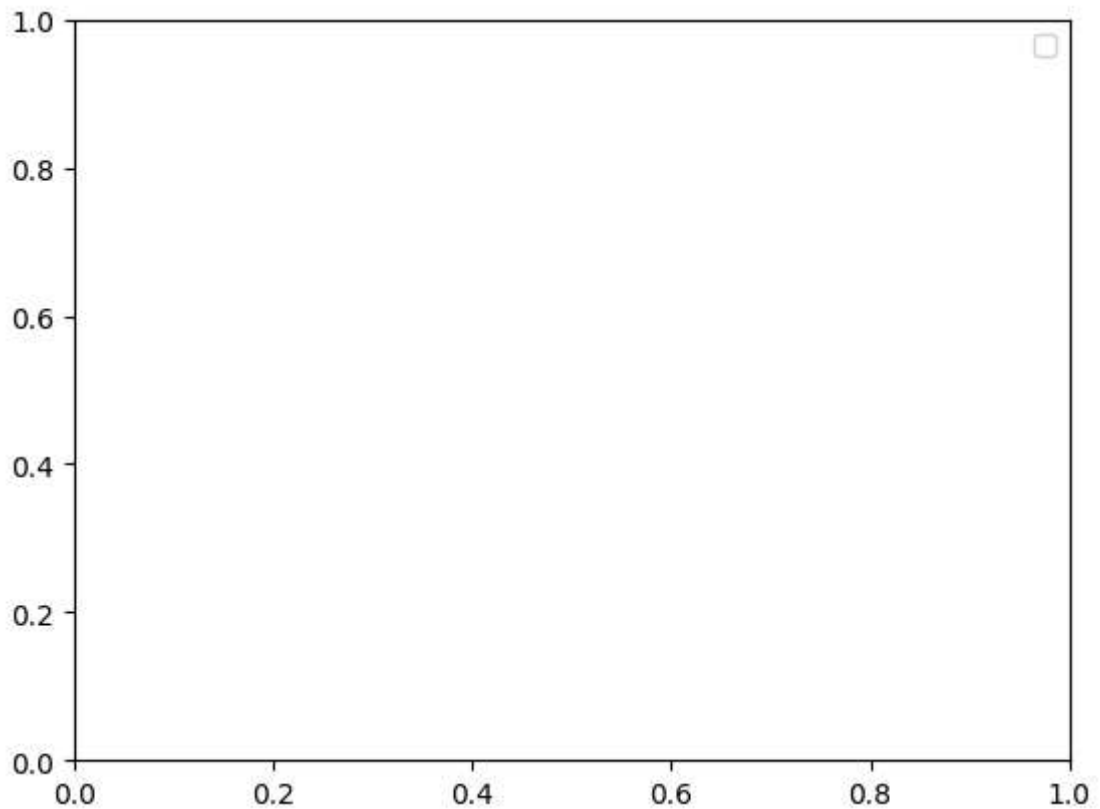
```
plt.plot(months, costs, marker='s', label='Costs', color='#F44336', linewidth
```

```
[<matplotlib.lines.Line2D at 0x7ba328703e30>]
```



```
plt.legend()
```

```
/tmp/ipykernel_1525/4061938096.py:1: UserWarning: No artists with labels found  
plt.legend()  
<matplotlib.legend.Legend at 0x7ba328a6b1a0>
```

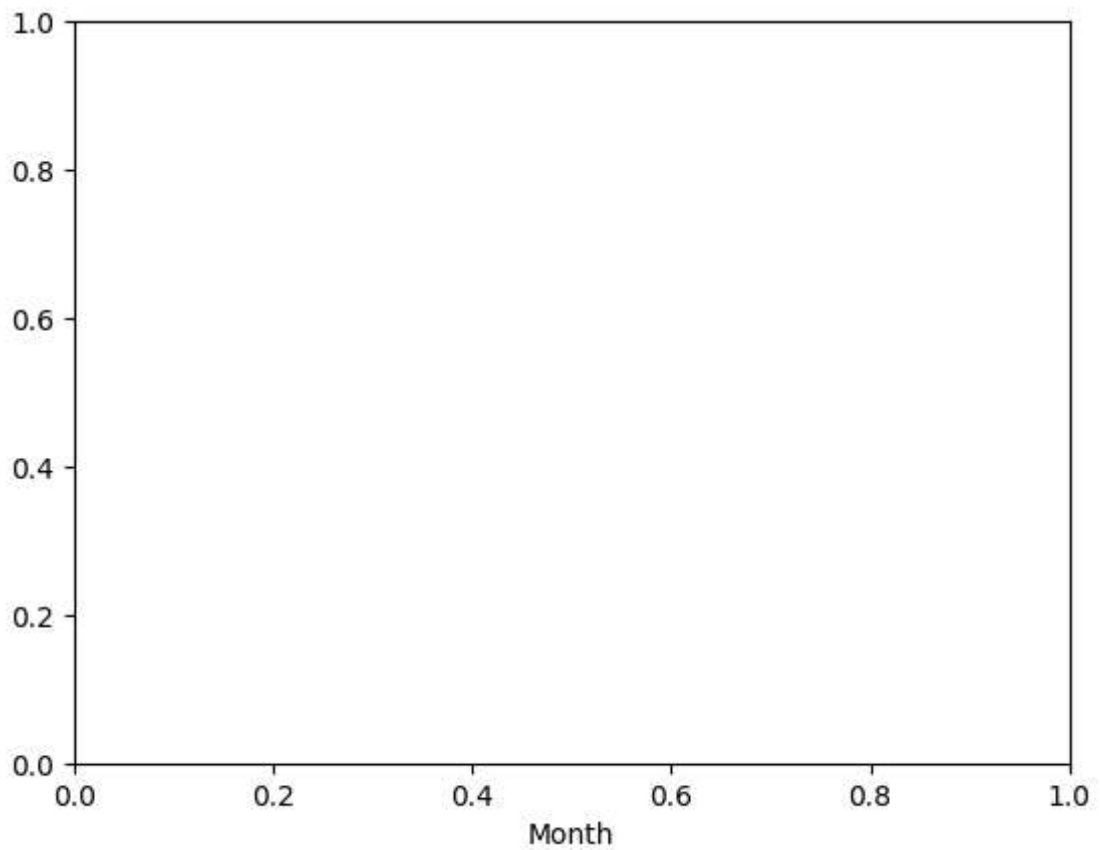


```
#used to provide title for this graph to understand purpose of graph  
plt.title('Monthly Sales vs Costs', fontsize=16, fontweight='bold')
```

[Show hidden output](#)

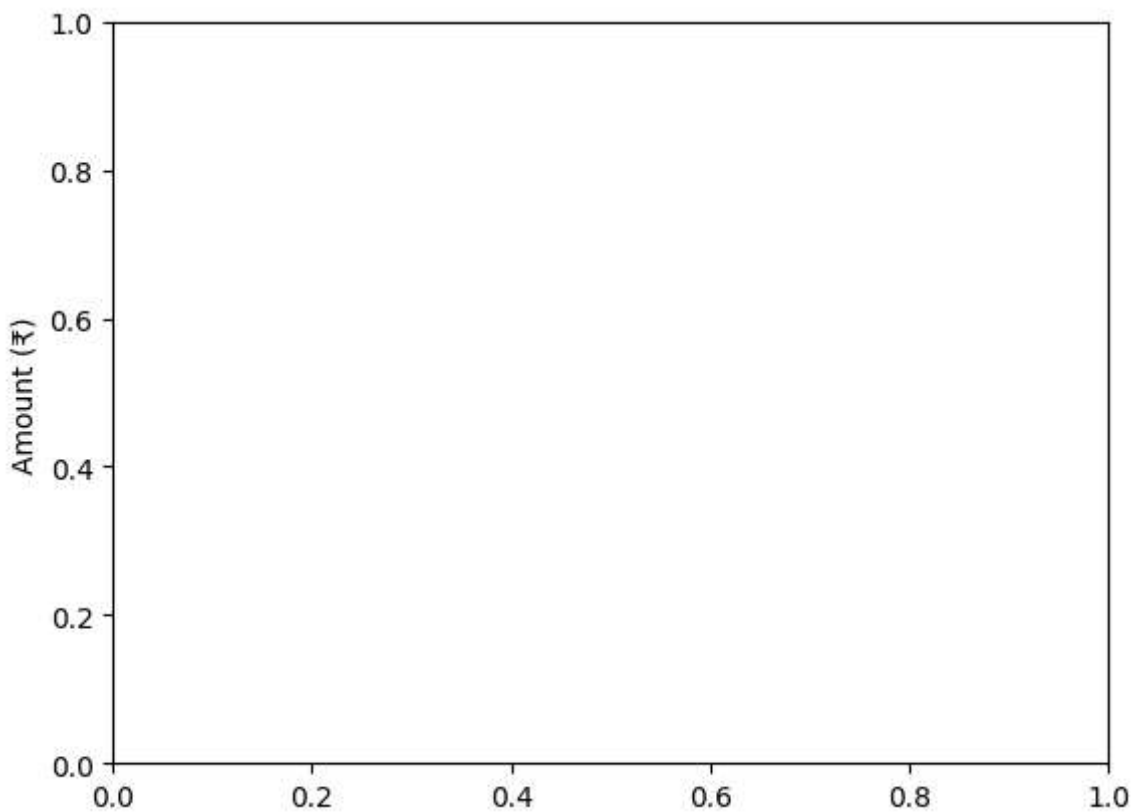
```
#adds label to x-axis  
plt.xlabel('Month')
```

```
Text(0.5, 0, 'Month')
```



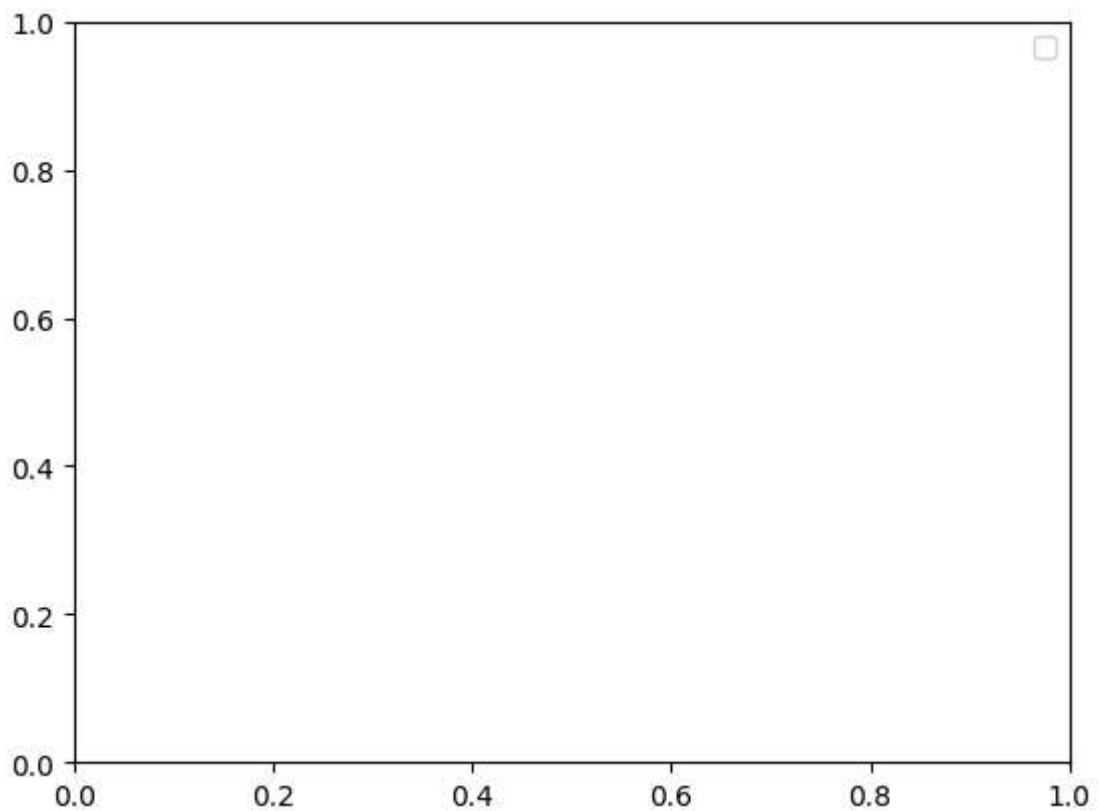
```
#adds lable to y-axis  
plt.ylabel('Amount (₹)')
```

```
Text(0, 0.5, 'Amount (₹)')
```

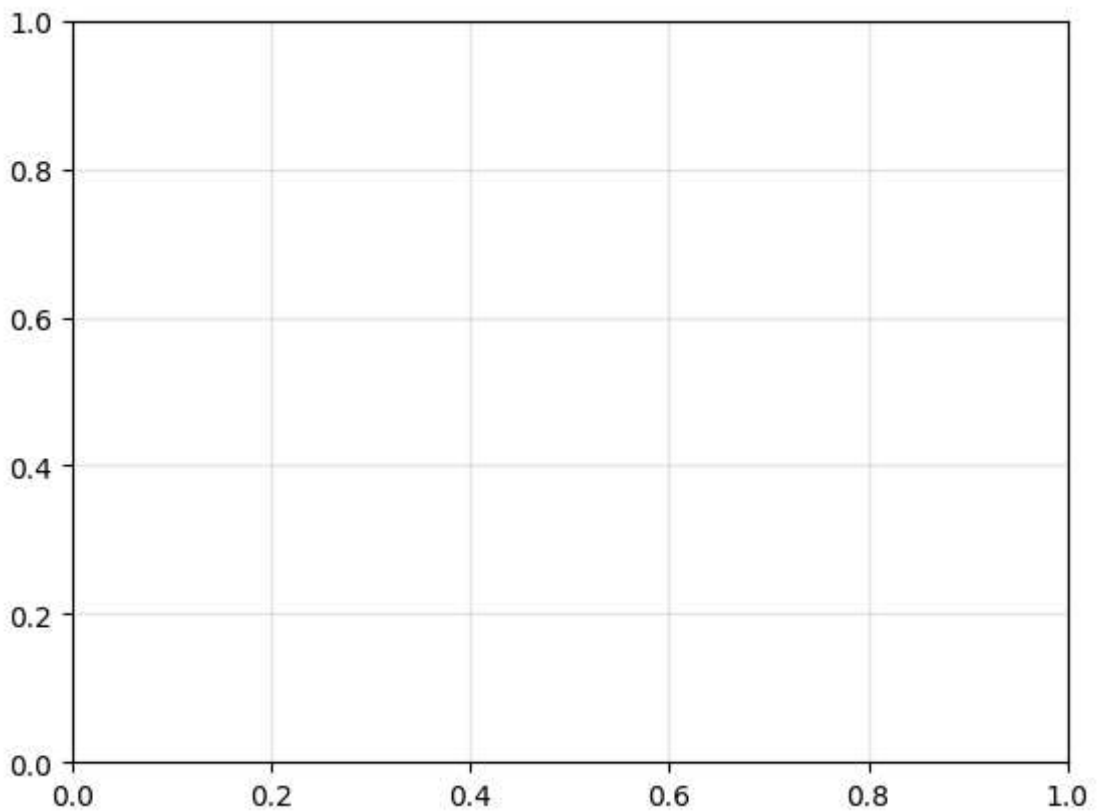


```
# Shows labels for plotted lines.  
plt.legend()
```

```
/tmp/ipykernel_1525/3779859436.py:2: UserWarning: No artists with labels found  
plt.legend()  
<matplotlib.legend.Legend at 0x7ba3287ab1a0>
```



```
# Adds background grid lines.  
# True=Turns grid ON.  
# alpha= controles tranparency  
plt.grid(True, alpha=0.3)
```



```
#Automatically adjusts spacing.  
plt.tight_layout()
```

<Figure size 640x480 with 0 Axes>

```
# Saves graph as image file.  
# DPI = Dots Per Inch  
plt.savefig('/content/sale_ct.png')
```

<Figure size 640x480 with 0 Axes>

```
plt.show()  
# Displays graph on screen.
```

```
# example code
```

```
import matplotlib.pyplot as plt  
  
months = ['Jan', 'Feb', 'Mar', 'Apr']  
sales = [100, 150, 120, 180]  
  
# Create figure  
plt.figure(figsize=(8,4))  
  
# Plot data  
plt.plot(months, sales,  
         marker='o',  
         color='blue',
```

```

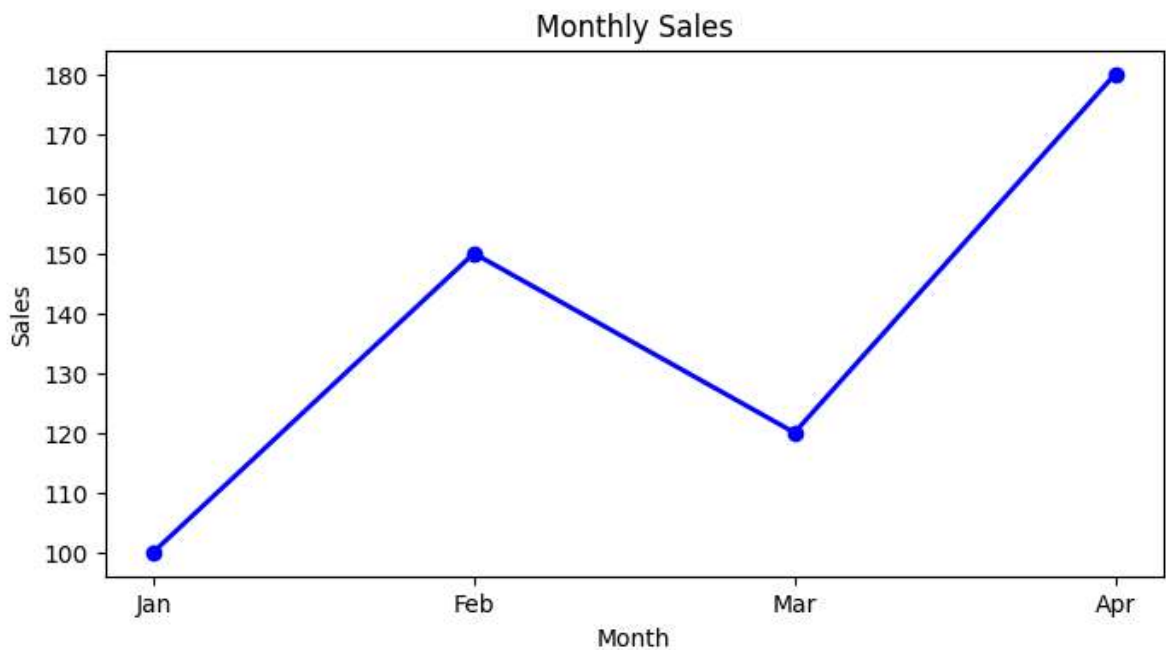
        linewidth=2)

# Labels and title
plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Sales")

# Save FIRST
plt.savefig('/content/sales_chart.png', dpi=300)

# THEN show
plt.show()

```



bar chart

```

cities = ['Mumbai', 'Delhi', 'Bangalore', 'Chennai', 'Hyderabad']
revenue = [4500, 3800, 5200, 2900, 3100]

fig, ax = plt.subplots(figsize=(10, 5)) # OOP style (preferred)

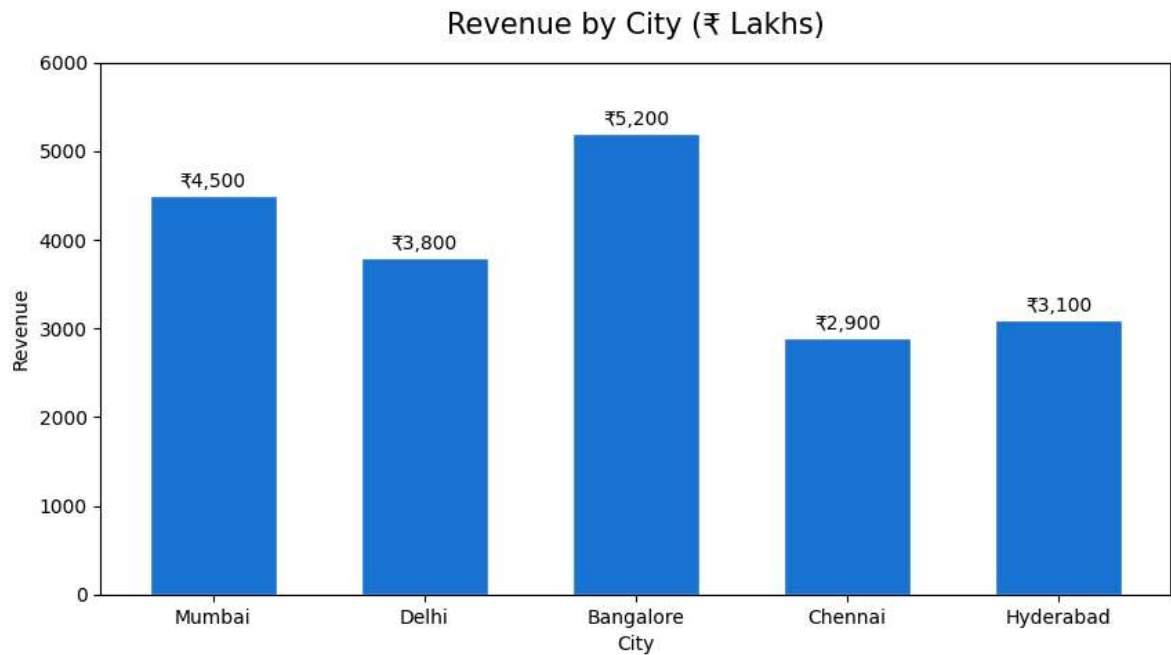
bars = ax.bar(cities, revenue, color='#1976D2', edgecolor='white', width=0.6)

# Add value labels ON TOP of each bar
for bar in bars:
    height = bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2, height + 50,
            f'₹{height:,}', ha='center', va='bottom', fontsize=10)

ax.set_title('Revenue by City (₹ Lakhs)', fontsize=15, pad=15)
ax.set_xlabel('City')
ax.set_ylabel('Revenue')
ax.set_ylim(0, 6000) # set y-axis range

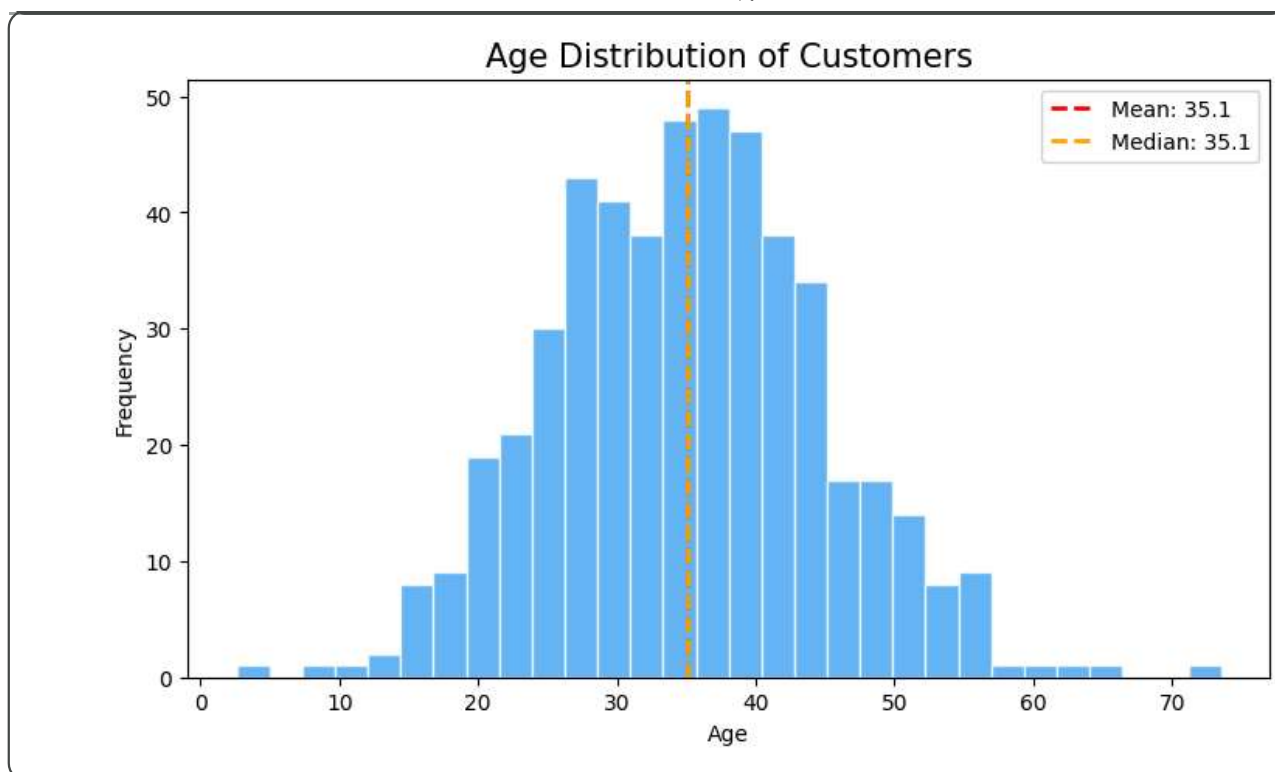
```

```
plt.savefig('revenue_bar.png', dpi=150, bbox_inches='tight')  
plt.show()
```



histogram

```
np.random.seed(42)  
ages = np.random.normal(35, 10, 500) # 500 ages, mean=35, std=10  
  
plt.figure(figsize=(9, 5))  
plt.hist(ages, bins=30, color='#42A5F5', edgecolor='white', alpha=0.8)  
  
# Add vertical mean line  
plt.axvline(ages.mean(), color='red', linestyle='--', linewidth=2, label=f'Mean')  
plt.axvline(np.median(ages), color='orange', linestyle='--', linewidth=2, label=f'Median')  
  
plt.title('Age Distribution of Customers', fontsize=15)  
plt.xlabel('Age')  
plt.ylabel('Frequency')  
plt.legend()  
plt.savefig('age_hist.png', dpi=150)  
plt.show()
```

Scatter Plot — Relationships

```

np.random.seed(0)
hours_studied = np.random.uniform(1, 10, 100)
exam_score    = 40 + 5 * hours_studied + np.random.normal(0, 5, 100)
study_group   = np.random.choice(['A', 'B', 'C'], 100)

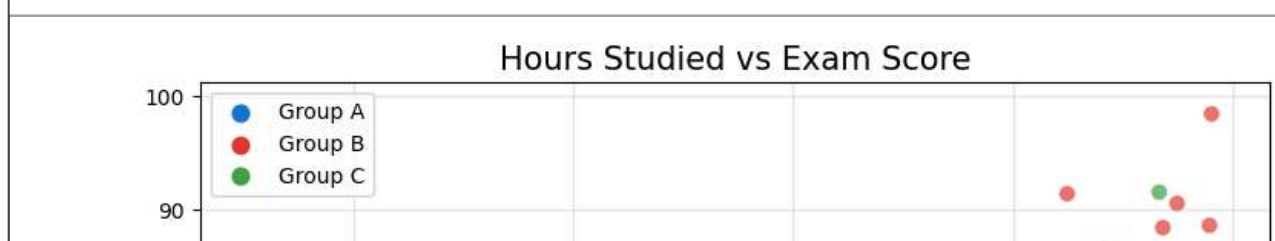
colors = {'A': '#1976D2', 'B': '#E53935', 'C': '#43A047'}
color_list = [colors[g] for g in study_group]

plt.figure(figsize=(9, 6))
scatter = plt.scatter(hours_studied, exam_score,
                      c=color_list, alpha=0.7, s=60, # s = marker size
                      edgecolors='white', linewidth=0.5)

# Legend for groups
for group, col in colors.items():
    plt.scatter([], [], c=col, label=f'Group {group}', s=60)

plt.title('Hours Studied vs Exam Score', fontsize=15)
plt.xlabel('Hours Studied')
plt.ylabel('Exam Score')
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig('study_scatter.png', dpi=150, bbox_inches='tight')
plt.show()

```




```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
```

```
sns.set_theme(style='whitegrid') # styles: whitegrid, darkgrid, white, dark, ticks

# Load the Titanic dataset (built-in!)
df = sns.load_dataset('titanic')
print(df.head())
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	\
0	0	3	male	22.0	1	0	7.2500	S	Third	
1	1	1	female	38.0	1	0	71.2833	C	First	
2	1	3	female	26.0	0	0	7.9250	S	Third	
3	1	1	female	35.0	1	0	53.1000	S	First	
4	0	3	male	35.0	0	0	8.0500	S	Third	

	who	adult_male	deck	embark_town	alive	alone
0	man	True	NaN	Southampton	no	False
1	woman	False	C	Cherbourg	yes	False
2	woman	False	NaN	Southampton	yes	True
3	woman	False	C	Southampton	yes	False
4	man	True	NaN	Southampton	no	True

Heatmap — Correlations between columns

showing data using colours

```
import seaborn as sns
import matplotlib.pyplot as plt

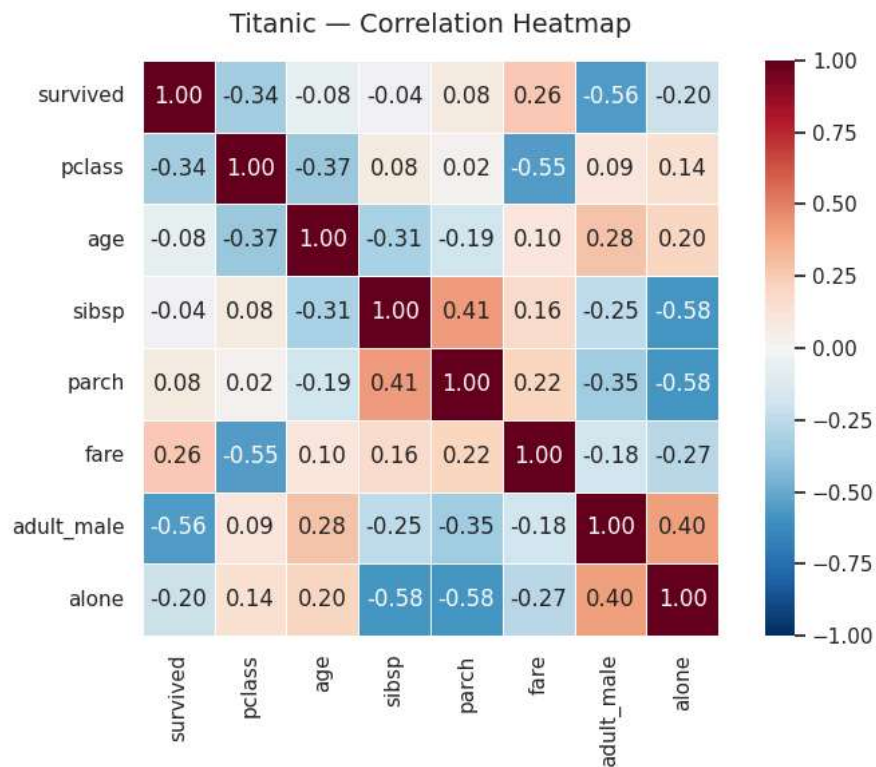
# Example: Titanic dataset (or your df)
# df = pd.read_csv("titanic.csv")

corr_matrix = df.corr(numeric_only=True)

plt.figure(figsize=(8, 6))

sns.heatmap(corr_matrix,
            annot=True,
            fmt='.2f',
            cmap='RdBu_r',
            vmin=-1, vmax=1,
            linewidths=0.5,
            square=True)

plt.title('Titanic - Correlation Heatmap', fontsize=14, pad=15)
plt.tight_layout()
plt.savefig('heatmap.png', dpi=150, bbox_inches='tight')
plt.show()
```



Boxplot — Spotting Outliers

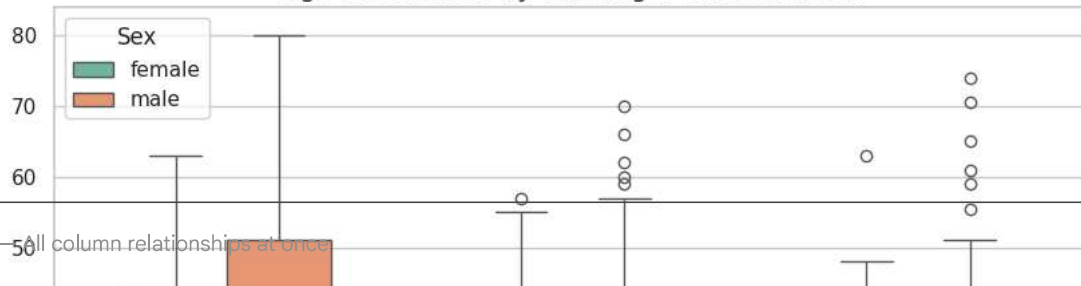
```
# Anatomy of a boxplot:
# Box top    = 75th percentile (Q3)
# Box bottom = 25th percentile (Q1)
# Line       = Median (Q2)
# Whiskers   = Q1 - 1.5*IQR to Q3 + 1.5*IQR
# Dots       = Outliers beyond whiskers

plt.figure(figsize=(10, 6))
sns.boxplot(data=df,
            x='pclass',          # 3 groups: 1st, 2nd, 3rd class
            y='age',              # numerical variable
            hue='sex',            # split by gender (2 colours)
            palette='Set2',
            width=0.6)

plt.title('Age Distribution by Passenger Class and Sex', fontsize=14)
plt.xlabel('Passenger Class')
plt.ylabel('Age')
plt.legend(title='Sex')
plt.savefig('boxplot.png', dpi=150, bbox_inches='tight')
plt.show()

# What to look for:
# - Dots = outliers (very old or very young passengers)
# - Box size = spread (3rd class has wider age range)
# - Median line position = skewness
```

Age Distribution by Passenger Class and Sex



Pairplot — All column relationships at once

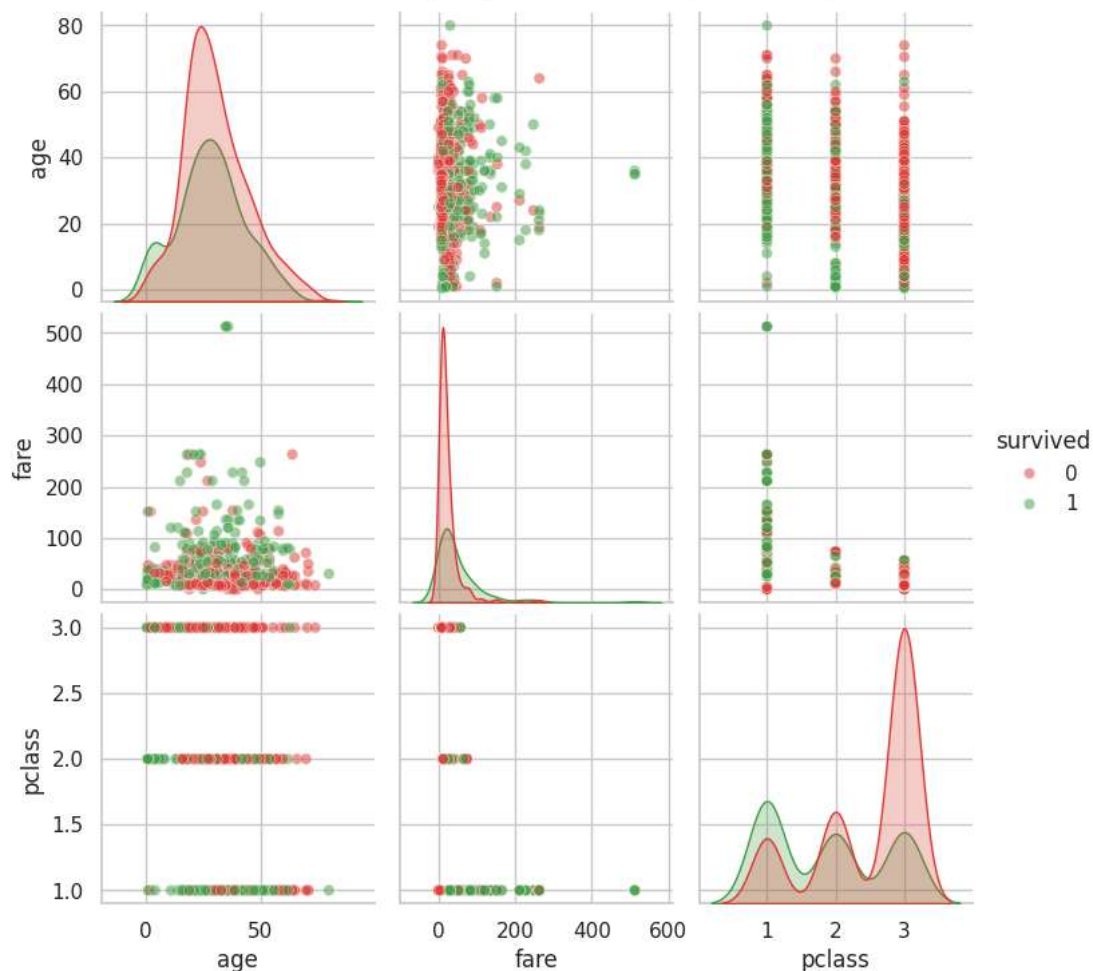
```
df_sub = df[['age', 'fare', 'survived', 'pclass']].dropna()

pairplot = sns.pairplot(df_sub,
                        hue='survived',          # colour by survival (0 or 1)
                        palette={0: '#E53935', 1: '#43A047'},
                        diag_kind='kde',          # diagonal: density curves
                        plot_kws={'alpha': 0.5},
                        height=2.5)

pairplot.fig.suptitle('Titanic — Pairplot (Green=Survived, Red=Died)',
                      y=1.02, fontsize=13)
pairplot.savefig('pairplot.png', dpi=150, bbox_inches='tight')
plt.show()

# Reading the pairplot:
# - Top row: age vs fare, age vs pclass, age vs survived
# - Diagonal: distribution of that single variable
# - Green clusters separate from red = good predictor of survival
```

Titanic — Pairplot (Green=Survived, Red=Died)



sns.set_theme(style=) Set style

sns.heatmap(corr) Heatmap

sns.boxplot(x=,y=,hue=) Boxplot

sns.pairplot(df) All pairs